

Memory v1

1. **One-shot references** (like brand guidelines, campaign docs, images → used only for this run).
2. **Long-term episodic memory** (agent's experience, past campaigns, its own learnings).

Here's a **solution architecture** that makes them work hand-in-hand without confusing RAG with memory:

◆ 1. Separate Knowledge Sources

- **Ephemeral Context (Refs)** → Things you upload or attach for *this run only* (brand guidelines PDF, sample campaign doc, moodboard images).
 - **Persistent Memory (Episodes)** → Things the agent has *experienced before* (past campaigns it created, mistakes it corrected, performance results).
-

◆ 2. Workflow Example – "Campaign Generation Agent"

Step 1: Input

- User uploads:
 - Brand guidelines PDF
 - Logo + 3 sample brand images
 - Tone-of-voice doc

Agent gets these as **ephemeral references**.

Step 2: Agent Task

- Agent generates:
 - Campaign copy
 - Image prompts for creative generation
 - Schedule (JSON / structured event graph)

Step 3: Store as Episodic Memory

- Save the final campaign plan (JSON + outputs) → **episode**.
- Add embeddings + metadata (client name, brand, date, success metrics later).
- Also store generated images → link them with the episode.

Now the agent has **learned experience** from this run.

◆ 3. Prompt Construction

When running the agent:

- Always provide **Ephemeral References**:

None

System: You are a campaign agent. For this run, here are the reference docs:

[Brand Guidelines PDF]

[Logo Image]

[Tone Doc]

- Append **Relevant Episodic Memories**:

None

Memory: Here are 2 past campaigns you created for this client.
Episode #12: "Diwali Sale 2024" → generated 10% CTR.

Episode #15: "Summer Drop 2025" → brand rejected images for being off-tone.

LLM now sees *both*, but knows ephemeral context is **authoritative for this run**.

◆ 4. Long-Term vs One-Time

- **Ephemeral refs** → thrown away after run.
 - **Episodes** → stay forever, keep growing.
-

◆ 5. Technical Implementation

- **Storage:**
 - Docs/images → in a vector DB (for retrieval) but flagged as *ephemeral* or *persistent*.
 - Episodes → JSON event graphs + embeddings + links to assets.
- **Runtime:**
 - Query vector DB for both:
 - All *ephemeral docs for this run*.
 - Top *episodic memories*.
 - Build prompt with separation markers:

None

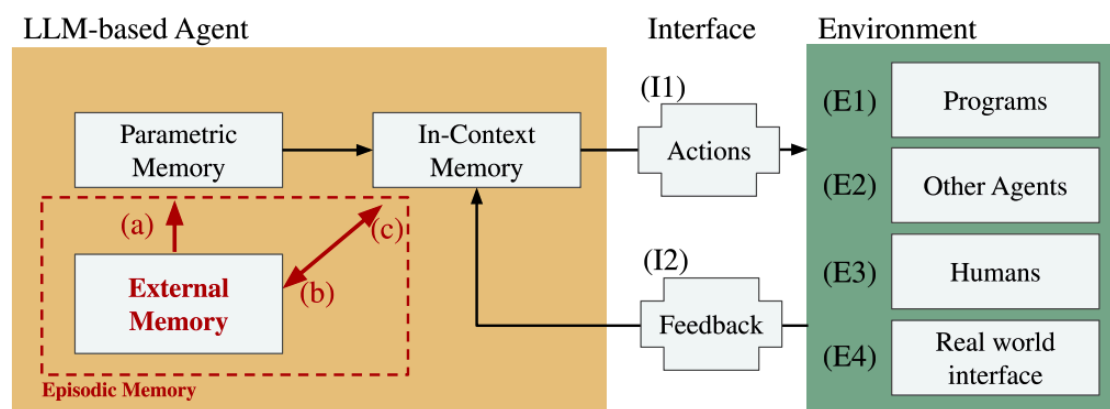
```
### Ephemeral Reference (Authoritative this run)
- Brand guidelines doc: ...
- Logo: (image embedding ref)
```

Episodic Memory (Past experiences, advisory)

- Past campaign #12: ...
- Past campaign #15: ...

✓ This way, your agent:

- Stays grounded in **brand guidelines for one run**.
- Still **remembers past lessons** for long-term improvement.
- Doesn't collapse into just RAG → because episodes are stored as structured JSON + outcomes, not just doc chunks.



Big Picture

Right now, LLMs use **three main memory types**:

- **In-context memory** (context window, short-term “working memory”).
- **External memory** (databases, vector stores → like RAG).
- **Parametric memory** (knowledge baked into weights).

Each of these covers *part* of what real memory does, but not all.

They're fragmented → no single mechanism lets an LLM **learn from experience over time** like humans.

Episodic memory is proposed as the glue:

A framework that combines all three, giving LLM agents a way to:

- Encode experiences → store them.
 - Retrieve them → when needed for reasoning.
 - Consolidate them → into stable long-term knowledge.
 - And be benchmarked fairly.
-

4.1 Encoding

RQ1: How do we **store experiences** from context into external memory?

- Today: we often just dump chunks into a vector DB (RAG).
- Future: more structured storage (like graphs, metadata, compressed embeddings).

RQ2: How do we **segment experiences** into “episodes”?

- Continuous input must be cut into meaningful chunks (like “this was a meeting” or “this was a game I played”).
 - Research: models can already segment text into meaningful events, and bundling events by “surprise” (novelty) improves long-term modeling.
-

4.2 Retrieval

RQ3: How do we pick the **right past episodes** at the right time?

- Current methods: prepend retrieved docs (RAG), use memory tokens, or adapt hidden states.
- Challenge: Don't overload the context window with irrelevant stuff.

RQ4: How do we **make retrieval smarter & faster** in long-context LLMs?

- Maybe integrate retrieval into the forward pass.
 - Or distill cross-architecture knowledge so memory acts more like built-in context, without huge costs.
-

4.3 Consolidation

RQ5: How do we **merge external memory back into parametric memory**?

- Otherwise, external DBs just keep growing forever.
- Ideas: context distillation, fine-tuning, knowledge editing.
- Challenge: avoid catastrophic forgetting (losing old knowledge).

This mirrors the **Complementary Learning Systems theory** in neuroscience:

- **Fast system (episodic memory)** = captures individual experiences.
 - **Slow system (semantic memory/parametric)** = consolidates into stable knowledge.
-

4.4 Benchmarks

RQ6: How do we **test episodic memory**?

- Must go beyond toy retrieval.
 - Need benchmarks where agents recall *when*, *where*, *how* events occurred (temporal order, contextualized recall).
 - And also show **improvement over long interactions** (true learning over time).
-

Takeaway

Episodic memory is positioned as the **missing piece** to unify:

- **Context window** (short-term).
- **External memory** (retrievable but brittle).
- **Parametric memory** (stable but hard to update).

If solved, LLM agents would:

- Learn from single interactions.
- Adapt continuously.
- Avoid forgetting.
- Operate more like *humans with memory* instead of *autocomplete machines*.

NEW ADD

◆ How Episodic Memory Fits into This Pipeline

1. Agent → Memory API

- When your campaign agent runs, it calls the **Memory API** with:
 - Raw text (logs, generated posts, brand guidelines reference, etc.)
 - Metadata (`user_id`, `agent_name`, `episode_id`)

👉 This is the “episode capture” step.
Each API call = one **experience chunk** to be stored.

2. Embedding Provider

- The raw episode text is converted to **embeddings**.

- Example: "Generated 3 posts for Sonali. 1 was approved. Engagement ↑15%."

👉 This allows **semantic recall** later, not just keyword matching.

3. Recall Pipeline

This is where episodic memory retrieval becomes powerful.

When the agent is about to start a new campaign:

- **Vector ANN Query:** Find past similar episodes (e.g., "Sonali + AI campaign").
- **Keyword Filter Contains:** Enforce `user_id`, `agent_name`, `episode_id` filters so memory doesn't leak across agents/users.
- **Pool Merge:** Combine both semantic matches + filtered results.
- **Re-rank:** Adjust results based on:
 - Recency (latest episodes preferred)
 - Usefulness (did feedback mark this as "helpful"?)
 - Engagement metrics (was this campaign successful?)
- **MMR Diversity:** Prevent only pulling "same-style" episodes → ensure variety in memory recall.

👉 Output is a **Distilled Snippet for Prompt**:

"Last time Sonali preferred technical AI insights. Campaigns using AI + Productivity hashtags had 15% more engagement. Meme-style posts performed poorly."

This is injected into the **agent's context** before it generates new output.

4. ChromaDB + Persistent Volume

- Episodic memories are stored in ChromaDB collections, partitioned by:
 - **tenant/project** (your platform)

- **agent_name** (CampaignGen vs TrendAnalyzer vs BrandGuard)
- **episode_id** (per run/session)
- Persistent Volume = long-term store so memory survives restarts.

👉 This gives you a **hybrid memory**: short-term recall + long-term episodic archive.

5. Feedback Loop

- After the campaign is launched, the system collects **feedback/outcomes**:
 - Engagement score
 - User rating (“this was good / bad”)
 - Explicit feedback (“avoid memes”)
- This updates the **usefulness score** of that episode inside ChromaDB metadata.
- Future queries → prefer high-rated, successful memories.

👉 Over time, the agent **learns from past experiences**, not just retrieves them.

◆ Example Flow with Your Social Campaign Agent

1. Episode Created

- Agent generates 3 LinkedIn posts.
- Episode stored:

```
JSON
{
  "user_id": "u123",
  "agent_name": "CampaignGen",
```

```
"episode_id": "ep15",  
  
"summary": "Generated 3 posts. Sonali approved 1. Engagement  
↑15%.",  
  
"feedback": "Positive"  
  
}
```

2.

Memory Recall (next run)

- Query: "Generate new campaign ideas for Sonali".
- Filters: { "user_id": "u123", "agent_name": "CampaignGen" }.
- Recall pipeline returns past episodes:
 - "Sonali approved technical posts."
 - "Avoid memes (negative feedback)."
 - "Best-performing hashtags: #AI #Productivity."

3. Agent Generates New Output

- Prompt includes:
 - Base instructions (brand guidelines)
 - Recalled episodic snippets (success/failure history)

4. Feedback Updates Memory

- New episode created → performance stored → scores updated.

✅ Net Result:

- Episodic memory = **experience storage**.
- Chroma pipeline = **structured retrieval**.

- Feedback loop = **learning from outcomes.**
- Memory API = **bridge between agent and memory system.**